

David Montgomery

DEVELOPER & AI SYSTEMS ARCHITECT

Centennial, CO 80112 | 303.335.7057 | dmontg@gmail.com

[GitHub](#) | [Medium](#) | [NPM](#) | [SecurityLens.io](#)

TECHNICAL PROFILE

Full-stack developer and AI systems architect specializing in RAG implementations, HIPAA-compliant APIs, developer tooling, and Model Context Protocol (MCP) integrations. Passionate about building production-ready systems that prioritize accessibility, comprehensive documentation, and user experience. Strong advocate for GUI-first design and making complex systems accessible to non-technical users.

CORE TECHNICAL SKILLS

Languages & Frameworks

- Python: FastAPI, LangChain, LangGraph, Flask, asyncio, multiprocessing
- JavaScript/TypeScript: Node.js, Express, React, Electron, Next.js
- Shell Scripting: Bash automation, system administration, CI/CD pipelines

AI & Machine Learning

- RAG Systems: Hybrid search (BM25 + dense embeddings), cross-encoder reranking, iterative retrieval
- Vector Databases: Qdrant, ChromaDB, Pinecone integration
- Embeddings: OpenAI text-embedding-3-large, Cohere embed-english-v3.0, sentence-transformers
- Agent Frameworks: LangGraph state machines, MCP tool development, agentic workflows

Infrastructure & DevOps

- Containerization: Docker, Docker Compose multi-profile deployments, container orchestration, Proxmox
- CI/CD: GitHub Actions, automated testing, deployment pipelines
- Databases: Redis, SQLite, PostgreSQL, MongoDB
- Cloud: AWS (S3, Lambda), Netlify deployment, serverless architectures

Security & Compliance

- HIPAA Compliance: PHI handling, HMAC verification, compliant logging, short-TTL tokens
- Authentication: API key management, JWT, OAuth2 flows, TBAC/RBAC, Hierarchical structures from Global to Multi-Tenant
- Penetration Testing: Nmap, Metasploit, Burp Suite, vulnerability assessment, Kali suite, Pentest MCP (creator)

FEATURED OPEN SOURCE PROJECTS

Faxbot – HIPAA-Compliant Self-Hosted Fax API Platform

github.com/DMontgomery40/Faxbot

First and only open-source, self-hostable fax platform with production-ready features, comprehensive security controls, and AI assistant integration.

Technical Architecture

- Backend: FastAPI with asyncio for concurrent webhook handling, Redis for state management, SQLite for metadata storage
- Multi-backend adapter pattern supporting cloud providers (Phaxio, Sinch, Documo, SignalWire) and self-hosted solutions (FreeSWITCH, SIP/Asterisk)
- Traits-first architecture with feature gating - UI and API dynamically adapt to active provider capabilities
- Canonical event model with normalized inbound/outbound events, status mapping, and standard error codes across all providers
- Clean provider adapter boundaries: `verify_webhook`, `parse_inbound`, `send`, `status`, `cancel` methods for easy extensibility

Security & Compliance

- HIPAA-aligned security controls: HMAC webhook signature verification, short-TTL API tokens, compliant logging architecture
- Zero PHI in logs - only document IDs and metadata surfaced for debugging while maintaining compliance
- Multi-key API authentication with scoped permissions, HTTPS enforcement, rate limiting
- Optional audit logging with configurable retention policies for enterprise compliance requirements

Developer Experience

- GUI-first Admin Console served by API - zero-configuration UI for provider management, diagnostics, and settings
- Identical SDKs for Node.js and Python published to npm and pip with comprehensive examples and type definitions
- Three MCP servers (stdio, HTTP, SSE) for AI assistant integration with Claude Code, enabling natural language fax operations
- Comprehensive MkDocs documentation site with dark mode, search, code highlighting, and live API reference
- Docker Compose deployment with health checks, automatic restarts, and multi-profile configuration (dev, prod, mcp)

Key Technical Achievements

- Reduced webhook processing latency by 40% through async/await optimization and connection pooling
- Implemented automatic provider failover with circuit breaker pattern for 99.9% uptime
- Built comprehensive test suite with 85% code coverage including integration tests for all supported providers

AGRO (Another Good RAG Option) – Production RAG Engine

github.com/DMontgomery40/agro-rag-engine

Sophisticated local-first RAG system for codebases with advanced retrieval techniques, cost optimization, and comprehensive developer tooling.

RAG Pipeline Architecture

- Hybrid search combining BM25 (keyword) with dense embeddings (OpenAI text-embedding-3-large, 3072-dimensional)
- Cross-encoder reranking using Cohere rerank-english-v3.0 with configurable multi-query (MQ) expansion (1-4 queries)
- LangGraph orchestration with state management, iterative retrieval loops (max 3 iterations), and confidence-based early stopping

- AST-aware code chunking using tree-sitter for Python/TypeScript/JavaScript with semantic boundary preservation
- Context compression with diversity-aware packing (token budget: 2500, directory diversity penalties)

Storage & Indexing

- Qdrant vector database for semantic search with 3072-dimensional embeddings and HNSW indexing
- BM25 implementation with custom tokenization and inverse document frequency (IDF) scoring
- Redis for LangGraph state persistence, configuration caching, and cross-branch index sharing
- Embedding cache with automatic pruning - tracks file content hashes to avoid redundant API calls (saves \$0.00013 per unchanged file)
- Incremental indexing with git hooks for automatic re-indexing on branch changes and commits

GUI Configuration Application

- Electron + React + TypeScript frontend with FastAPI backend on port 8765
- Dynamic cost calculator with real-time updates showing per-search, daily, and monthly costs with pie chart breakdowns
- Auto-profile engine with hardware detection (CPU, RAM, GPU, local LLMs) and budget-optimized configuration suggestions
- Profile system with preset configurations: Urgent/Production (\$100/day), Regular/Balanced (\$20/day), Budget (\$10/day), Offline/Local (\$0/day)
- Live settings application - no .env file editing required, zero process restarts, validation prevents invalid configs
- Embedded VS Code server integration with lifecycle management (start, stop, health checks) directly from GUI

Advanced Features

- LangSmith trace integration with live preview, embedding, and sharing capabilities for debugging RAG pipelines
- Evaluation harness with golden question sets, automated scoring (faithfulness, groundedness, precision)
- Keyword extraction using LLM-based metadata enrichment (MLX, Ollama, OpenAI backends)
- Cross-branch indexing with shared index directory (out.noindex-shared) for consistent results across git branches
- MCP server with 4 tools (rag_answer, rag_search, netlify_deploy, web_get) in three transports: stdio, HTTP, SSE

Performance Optimization

- Cost optimization: reduced daily spend from \$40.88 (MQ=4) to \$12.38 (MQ=1) while maintaining quality - 70% savings
- Fixed near-zero confidence scores (0.001%) by rebuilding corrupt chunk indices - improved top-1 relevance to 1.80 (180%)
- Embedding cache with hash-based deduplication - 177 cached embeddings reduced to 142 with automatic pruning
- Lazy loading strategy for document hydration to minimize memory footprint during retrieval

agentspec – AI-Safe Docstring Generator

github.com/DMontgomery40/agentspec

CLI tool that generates comprehensive docstrings with explicit guardrails to prevent AI agents from making destructive code changes.

Core Architecture

- Three-pass architecture ensuring LLM never sees deterministic metadata to prevent hallucination and anti-suggestion effects
- Pass 1: Collectors extract signatures, dependencies, git history from AST and static analysis
- Pass 2: LLM generates reasoning (what, why, rationale, guardrails) completely blind to Pass 1 data
- Pass 3: Formatter combines LLM reasoning + deterministic metadata into PEP 257/258 compliant docstrings

Guardrail Generation

- Explicit DO NOT / NEVER / ALWAYS statements with reasoning to prevent common AI agent mistakes
- Examples: 'DO NOT change gpt-5 to gpt-4o-mini - gpt-5 exists as of April 2025', 'NEVER remove rate limiting - prevents 429 errors at scale'
- Minimum 3 specific actionable statements per function with consequence explanations

Features & Tooling

- Intelligent linting: validates YAML syntax, required fields, minimum verbosity (configurable thresholds)
- Multi-provider support: OpenAI (gpt-5), Anthropic (claude-sonnet-4-5), Ollama (llama3.2), with unified base system prompt
- Profile system: production (OpenAI gpt-5, terse), fast (Claude Haiku), quality (Claude Sonnet), local (Ollama)
- Auto-detect: per-file language detection with automatic parser/formatter switching for mixed .py/.ts/.js codebases
- Modes: verbose (full agent guardrails) vs terse (human-readable concise docs) with --update-existing flag

Technical Implementation

- AST parsing with language-specific extractors for Python, TypeScript, JavaScript using tree-sitter
- Git integration: extracts commit history, authorship, and change frequency for context
- Dependency graph analysis: tracks function calls, called_by relationships, config/env var usage
- Format-agnostic: supports Google-style, NumPy-style, Sphinx docstrings with custom sections (Rationale, Guardrails, Dependencies)

Additional Technical Projects

MCP Server Ecosystem

- DeepSeek MCP Server: Natural language configuration with intelligent request mapping, published to npm with 3+ stars
- Pentest MCP: Professional penetration testing toolkit with nmap, gobuster, nikto, JtR, hashcat - NOT for educational purposes
- 3D Printer MCP: Multi-printer API integration (Orca, Bambu, OctoPrint, Klipper, Duet, Repetier, Prusa, Creality) with STL operations
- Canvas LMS MCP: 54 tools for course management, assignments, enrollments, grades - comprehensive educational platform integration

Infrastructure & Utilities

- Intel NPU Top: Real-time Intel Meteor Lake NPU monitoring via sysfs on Linux - 200 lines, zero dependencies, Docker support
- Docs Autopilot: LLM-assisted MkDocs documentation generator with auto-deploy to GitHub Pages via Netlify proxy
- MCP Security Scanner: JavaScript vulnerability scanner with GitHub Pages deployment and plugin architecture

Frigate NVR Contributions

- GUI configuration tools emphasizing accessibility improvements for non-technical users
- Focus on reducing complexity of video surveillance system configuration through visual interfaces

TECHNICAL PHILOSOPHY & APPROACH

- Accessibility-First: Design for users with ADHD/dyslexia - prefer GUI over config files, complete solutions over theoretical implementations
- Production-Ready: Comprehensive error handling, security considerations, extensive documentation - not just proof-of-concepts
- Developer Experience: Identical SDKs across languages, exhaustive examples, GUI-first admin tools, zero-config deployments
- Principles: DRY (Don't Repeat Yourself), KISS (Keep It Simple), YAGNI (You Aren't Gonna Need It)
- Architecture: Deterministic metadata invisible to LLMs, traits-first feature gating, clean adapter boundaries

EDUCATION & CERTIFICATIONS

- University of Denver (Oct 2024 – Present): Continuing Education in AI for Cybersecurity
- M.S. in Organizational Development, Colorado State University
- B.A. in Political Science, Metropolitan State University of Denver
- CompTIA: Security+, Network+, Linux+, A+

GitHub: github.com/DMontgomery40 | Full project portfolio available